

IET ASSIGNMENT – 1

URBAN COMPLAINT MANAGEMENT AND SERVICE REQUEST SYSTEM

Course: DSA0108 – Object Oriented Programming with C++
Implementation: C++ Console Application

1. Introduction

The Urban Complaint Management and Service Request System is a menu-driven, object-oriented C++ application designed to manage common civic complaints such as road damage, water supply problems and waste-management issues. The system allows citizens to be registered, complaints to be recorded, complaint priority to be calculated, departments to be assigned, statuses to be updated, complaints to be compared and service-performance information to be reported.

The implementation focuses on the object-oriented programming requirements of the assignment. It uses encapsulation, constructors, a copy constructor, a destructor, inheritance, virtual functions, runtime polymorphism, pointers, a static member, operator overloading and a collection of dynamically created complaint objects.

2. Objectives

- Register citizens and maintain their basic details.
- Register and categorize urban complaints.
- Calculate complaint priority using severity, impact, urgency and complaint type.
- Automatically recommend the appropriate service department.
- Update and track complaint status.
- Compare complaints based on priority using operator overloading.
- Generate a service-performance report.
- Demonstrate major C++ object-oriented programming concepts in a practical application.

3. System Features

Feature	Description
Citizen Registration	Stores citizen ID and name.
Complaint Registration	Creates a complaint object for Road, Water or Waste issues.
Smart Priority	Computes a priority score from severity, affected people, urgency and complaint type.
Automatic Department	Maps complaint type to a recommended department.
Status Tracking	Supports Pending, In Progress and Resolved states.

Complaint Comparison	Compares priority scores using overloaded operators.
Service Report	Displays total, pending, in-progress and resolved complaints.

4. Class Design

Class	Responsibility
Citizen	Encapsulates citizen ID and name; demonstrates constructors and copying.
Complaint	Base class containing common complaint data, priority calculation, status and operators.
RoadComplaint	Derived class for road-related complaints.
WaterComplaint	Derived class for water-supply complaints.
WasteComplaint	Derived class for waste-management complaints.

Inheritance hierarchy:

Complaint

/

|

\

Road Water Waste

5. OOP Concepts Implemented

Concept	Implementation
Encapsulation	Citizen data is private; complaint data is protected and accessed through member functions.
Default Constructor	Initializes objects with default values.
Parameterized Constructor	Creates complaint objects using supplied citizen,

	description and priority inputs.
Copy Constructor	Copies the data of an existing Citizen or Complaint object.
Destructor	Complaint has a virtual destructor; dynamically created complaint objects are deleted before program termination.
Inheritance	RoadComplaint, WaterComplaint and WasteComplaint inherit from Complaint.
Virtual Functions	Derived classes override type(), dept() and typeScore().
Runtime Polymorphism	Complaint pointers refer to different derived complaint objects.
Pointers	Complaint* is used for dynamic object creation and lookup.
Static Member	Complaint::total stores the total number of registered complaints.
Operator Overloading	operator> compares priority and operator<< formats complaint output.
Vector	Vectors store multiple citizens and complaint pointers.

6. Smart Priority Calculation

The system adds a simple decision-making feature instead of assigning a fixed priority. The score is calculated using severity, population impact, urgency and the complaint type score.

Priority Score = Severity + Impact Score + Urgency + Type Score

The impact score increases when more people are affected. Water complaints receive a higher type score in the current implementation because interruption of water supply is treated as more urgent. The resulting score is classified as LOW, MEDIUM, HIGH or CRITICAL.

7. Menu-Driven Workflow

1. Register Citizen
2. Register Complaint

3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

8. Pseudocode

START

Create collections for citizens and complaints.

Display the main menu.

REPEAT

 Read menu choice.

 IF Register Citizen

 Read citizen details.

 Create Citizen object.

 Store citizen.

 ELSE IF Register Complaint

 Find citizen.

 Read complaint type and priority inputs.

 Create the appropriate derived Complaint object.

 Calculate priority.

 Store complaint.

 ELSE IF View Complaints

 Display all complaint objects.

 ELSE IF Assign Department

 Find complaint.

 Assign recommended department.

ELSE IF Update Status

Find complaint.

Change its status.

ELSE IF Compare Complaints

Find two complaints.

Compare their priority scores using operator>.

ELSE IF Service Report

Count complaints by status.

Display statistics.

UNTIL Exit.

Delete dynamically allocated complaint objects.

END

Code:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
// ----- CITIZEN -----
```

```
class Citizen {
```

```
    int id;
```

```
    string name;
```

```
public:
```

```
    Citizen() : id(0), name("Unknown") {}
```

```
    Citizen(int i, string n) : id(i), name(n) {}
```

```

Citizen(const Citizen &c) {
    id = c.id;
    name = c.name;
}

int getId() { return id; }
string getName() { return name; }
};

// ----- BASE COMPLAINT -----
class Complaint {
protected:
    int id, severity, affected, urgency, score;
    Citizen citizen;
    string description, status, department;

public:
    static int total;

    Complaint() :
        id(0), severity(1), affected(1), urgency(1),
        score(0), status("Pending"),
        department("Not Assigned") {}

    Complaint(int i, Citizen c, string d,
        int s, int a, int u)
        : id(i), citizen(c), description(d),
        severity(s), affected(a), urgency(u),
        status("Pending"), department("Not Assigned") {
        calculatePriority();
        total++;
    }
}

```

```
Complaint(const Complaint &c) {
```

```
    id = c.id;
```

```
    citizen = c.citizen;
```

```
    description = c.description;
```

```
    severity = c.severity;
```

```
    affected = c.affected;
```

```
    urgency = c.urgency;
```

```
    score = c.score;
```

```
    status = c.status;
```

```
    department = c.department;
```

```
}
```

```
virtual ~Complaint() {}
```

```
virtual string type() { return "General"; }
```

```
virtual string dept() { return "General Services"; }
```

```
virtual int typeScore() { return 1; }
```

```
void calculatePriority() {
```

```
    int impact = (affected >= 50) ? 5 :
```

```
        (affected >= 20) ? 4 :
```

```
        (affected >= 5) ? 2 : 1;
```

```
    score = severity + impact + urgency + typeScore();
```

```
}
```

```
string level() {
```

```
    if (score >= 15) return "CRITICAL";
```

```
    if (score >= 11) return "HIGH";
```

```
    if (score >= 7) return "MEDIUM";
```

```
    return "LOW";
```

```
}
```



```
void assign() {
    department = dept();
}
```

```
void update(string s) {
    status = s;
}
```

```
int getId() { return id; }
int getScore() { return score; }
string getStatus() { return status; }
```

// Operator overloading

```
bool operator>(Complaint &c) {
    return score > c.score;
}
```

```
friend ostream& operator<<(ostream &out,
```

```
    Complaint &c) {
```

```
    out << left << setw(6) << c.id
```

```
        << setw(12) << c.type()
```

```
        << setw(15) << c.status
```

```
        << setw(20) << c.department
```

```
        << setw(8) << c.score
```

```
        << c.level();
```

```
    return out;
```

```
}
```

```
};
```

```
int Complaint::total = 0;
```

// ----- ROAD -----

```
class RoadComplaint : public Complaint {
```

public:

```
RoadComplaint(int i, Citizen c, string d,  
              int s, int a, int u)  
    : Complaint(i, c, d, s, a, u) {}
```

```
string type() override {  
    return "Road";  
}
```

```
string dept() override {  
    return "Road Maintenance";  
}
```

```
int typeScore() override {  
    return 3;  
}  
};
```

// ----- WATER -----

```
class WaterComplaint : public Complaint {  
public:
```

```
    WaterComplaint(int i, Citizen c, string d,  
                  int s, int a, int u)  
        : Complaint(i, c, d, s, a, u) {}
```

```
string type() override {  
    return "Water";  
}
```

```
string dept() override {  
    return "Water Supply";  
}
```

```

    int typeScore() override {
        return 5;
    }
};

```

// ----- WASTE -----

```

class WasteComplaint : public Complaint {
public:
    WasteComplaint(int i, Citizen c, string d,
                    int s, int a, int u)
        : Complaint(i, c, d, s, a, u) {}

```

```

    string type() override {
        return "Waste";
    }

```

```

    string dept() override {
        return "Sanitation";
    }

```

```

    int typeScore() override {
        return 2;
    }
};

```

// ----- FIND COMPLAINT -----

```

Complaint* find(vector<Complaint*> &v, int id) {

    for (Complaint *c : v)
        if (c->getId() == id)
            return c;

```

```
    return nullptr;
}
```

```
// ----- MAIN -----
```

```
int main() {
```

```
    vector<Complaint*> complaints;
```

```
    vector<Citizen> citizens;
```

```
    int citizenID = 101;
```

```
    int complaintID = 1001;
```

```
    int choice;
```

```
    do {
```

```
        cout << "\n\n===== URBAN COMPLAINT SYSTEM =====\n";
```

```
        cout << "1. Register Citizen\n";
```

```
        cout << "2. Register Complaint\n";
```

```
        cout << "3. View Complaints\n";
```

```
        cout << "4. Assign Department\n";
```

```
        cout << "5. Update Status\n";
```

```
        cout << "6. Compare Complaints\n";
```

```
        cout << "7. Service Report\n";
```

```
        cout << "8. Exit\n";
```

```
        cout << "Choice: ";
```

```
        cin >> choice;
```

```
    // REGISTER CITIZEN
```

```
    if (choice == 1) {
```

```
        string name;
```

```
        cout << "Citizen Name: ";
```

```

cin >> name;

citizens.push_back(
    Citizen(citizenID++, name));

cout << "Citizen registered.\n";
}

// REGISTER COMPLAINT
else if (choice == 2) {

    if (citizens.empty()) {
        cout << "Register citizen first.\n";
        continue;
    }

    int cid, type, s, a, u;
    string d;

    cout << "Citizen ID: ";
    cin >> cid;

    Citizen *cptr = nullptr;

    for (Citizen &c : citizens)
        if (c.getId() == cid)
            cptr = &c;

    if (!cptr) {
        cout << "Citizen not found.\n";
        continue;
    }

    cout << "\n1.Road 2.Water 3.Waste\n";

```

```
cout << "Type: ";
```

```
cin >> type;
```

```
cout << "Description: ";
```

```
cin >> d;
```

```
cout << "Severity (1-5): ";
```

```
cin >> s;
```

```
cout << "Affected people: ";
```

```
cin >> a;
```

```
cout << "Urgency (1-5): ";
```

```
cin >> u;
```

```
Complaint *p = nullptr;
```

```
if (type == 1)
```

```
    p = new RoadComplaint(  
        complaintID++, *cptr, d, s, a, u);
```

```
else if (type == 2)
```

```
    p = new WaterComplaint(  
        complaintID++, *cptr, d, s, a, u);
```

```
else if (type == 3)
```

```
    p = new WasteComplaint(  
        complaintID++, *cptr, d, s, a, u);
```

```
else {
```

```
    cout << "Invalid type.\n";
```

```
    continue;
```

```
}
```

```

complaints.push_back(p);

cout << "\nComplaint Registered!";
cout << "\nPriority Score: "
    << p->getScore();

cout << "\nPriority Level: "
    << p->level() << endl;
}

// VIEW
else if (choice == 3) {

    cout << "\nID    Type    Status"
        << "    Department    Score Level\n";

    cout << "-----\n";

    for (Complaint *c : complaints)
        cout << *c << endl;
}

// ASSIGN
else if (choice == 4) {

    int id;
    cout << "Complaint ID: ";
    cin >> id;

    Complaint *c = find(complaints, id);

    if (c) {
        c->assign();
        cout << "Department assigned: "

```

```

        << c->dept() << endl;
    }
    else
        cout << "Complaint not found.\n";
}

// STATUS
else if (choice == 5) {

    int id, s;

    cout << "Complaint ID: ";
    cin >> id;

    Complaint *c = find(complaints, id);

    if (c) {

        cout << "1.Pending 2.In Progress 3.Resolved\n";
        cin >> s;

        if (s == 1) c->update("Pending");
        else if (s == 2) c->update("In Progress");
        else if (s == 3) c->update("Resolved");

        cout << "Status updated.\n";
    }
    else
        cout << "Complaint not found.\n";
}

// COMPARE
else if (choice == 6) {

```



```

int a, b;

cout << "First Complaint ID: ";
cin >> a;

cout << "Second Complaint ID: ";
cin >> b;

Complaint *c1 = find(complaints, a);
Complaint *c2 = find(complaints, b);

if (c1 && c2) {

    if (*c1 > *c2)
        cout << a << " has higher priority.\n";

    else if (*c2 > *c1)
        cout << b << " has higher priority.\n";

    else
        cout << "Equal priority.\n";
}
else
    cout << "Complaint not found.\n";
}

// REPORT
else if (choice == 7) {

    int pending = 0;
    int progress = 0;
    int resolved = 0;

    for (Complaint *c : complaints) {

```

```

        if (c->getStatus() == "Pending")
            pending++;

        else if (c->getStatus() == "In Progress")
            progress++;

        else if (c->getStatus() == "Resolved")
            resolved++;
    }

    cout << "\n===== SERVICE REPORT =====\n";

    cout << "Total Complaints : "
        << Complaint::total << endl;

    cout << "Pending      : "
        << pending << endl;

    cout << "In Progress    : "
        << progress << endl;

    cout << "Resolved      : "
        << resolved << endl;
}

} while (choice != 8);

// Destructor / memory cleanup
for (Complaint *c : complaints)
    delete c;

cout << "\nProgram ended successfully.\n";

```

```
return 0;
```

```
}
```

```
Choice: 5
Complaint ID: 1001
1.Pending 2.In Progress 3.Resolved
2
Status updated.

===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
Choice: 5
Complaint ID: 1002
1.Pending 2.In Progress 3.Resolved
3
Status updated.

===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints

Choice: 2
Citizen ID: 102

1.Road 2.Water 3.Waste
Type: 1
Description: roadDamage
Severity (1-5): 3
Affected people: 20
Urgency (1-5): 3

Complaint Registered!
Priority Score: 11
Priority Level: HIGH

===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status

Choice: 3

ID      Type      Status      Department      Score Level
-----
1001    Water      Pending      Not Assigned      16      CRITICAL
1002    Road        Pending      Not Assigned      11      HIGH

===== URBAN COMPLAINT SYSTEM =====
1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit
Choice: 4
Complaint ID: 1001
Department assigned: Water Supply
```

1.Road 2.Water 3.Waste
Type: 2
Description: Watersupply
Severity (1-5): 5
Affected people: 100
Urgency (1-5): 5

Complaint Registered!
Priority Score: 16
Priority Level: CRITICAL

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

Choice:

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

Choice: 1

Citizen Name: Ravi
Citizen registered.

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

Choice: 6

First Complaint ID: 1001
Second Complaint ID: 1002
1001 has higher priority.

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status

7. Service Report
 8. Exit
- Choice: 1
Citizen Name: Priya
Citizen registered.

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

Choice: 2

Citizen ID: 101

7. Service Report

8. Exit

Choice: 7

===== SERVICE REPORT =====

Total Complaints : 2
Pending : 0
In Progress : 1
Resolved : 1

===== URBAN COMPLAINT SYSTEM =====

1. Register Citizen
2. Register Complaint
3. View Complaints
4. Assign Department
5. Update Status
6. Compare Complaints
7. Service Report
8. Exit

9. Reflection and Learning Outcomes

This project helped demonstrate how C++ OOP concepts can be combined to solve a real-world civic-service problem. The use of a base Complaint class and derived complaint types reduces repetition and allows runtime polymorphism. Constructors make object creation organized, while the destructor and delete operations demonstrate memory cleanup.

The smart priority calculation was selected as a practical enhancement because it makes complaint handling more meaningful than simply storing records. A major learning point was understanding how a base-class pointer can refer to different derived objects and invoke overridden functions at runtime.

10. SDG Relevance

The assignment maps the project to SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 16 – Peace, Justice and Strong Institutions; and SDG 12 – Responsible Consumption and Production.

The strongest connection is SDG 11 because the application provides a structured way to register, prioritize and track urban service complaints, supporting better management of community issues.

11. Conclusion

The Urban Complaint Management and Service Request System provides a compact but feature-rich C++ implementation for civic complaint handling. It combines a menu-driven interface with inheritance, runtime polymorphism, constructors, destructor, pointers, static members, operator overloading and smart priority calculation. The system can be extended in the future with additional complaint types, persistent file storage, authentication or a graphical/web interface.